

98.1. La fonction `paletteLutins()` — les nuances des lutins

`#include <paletteLutins>` : `paletteLutins(0 ou 1, n0, n1, n2, n3)` choisit les nuances d'une des deux palettes des lutins (Game Boy d'origine).

1. Ce qu'il faut comprendre

Ce qu'elle fait : sur la Game Boy d'origine, les lutins ont **deux** palettes de nuances. `paletteLutins(p, n0, n1, n2, n3)` règle la palette `p` (0 ou 1). La teinte 0 reste transparente.

La ligne à écrire : `#include <paletteLutins>`.

Ses arguments : la palette, 0 ou 1, puis quatre nuances de 0 à 3.

Ce qu'elle coûte : environ **95 octets** avec la mise en place des lutins.

Essaie : `paletteLutins(0, 0, 1, 1, 1);` : le A devient gris clair.

On retrouve `paletteLutins` dans 2 leçons

[99](#), [100](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **219 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	47 o	← la console, à l'allumage – la mise en route de la console
main()	37 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
RangerLutins	11 o	← démarrage – range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles	31 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
Tuiles	16 o	← CopierTuiles – 1 lettre sur 43
DonneesTransfert	10 o	← InstallerTransfert – la petite routine qui copie les lutins

Lettres gravées : A

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `EffacerCarte`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `Notes`

Fonctions du langage appelées :

<code>sprite()</code>	× 1	23 o sur place, dans <code>main()</code>
<code>image()</code>	× 1	6 o sur place, dans <code>main()</code> + <code>AttendreImage</code> (28 o)
<code>paletteLutins()</code>	× 1	4 o sur place, dans <code>main()</code>

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

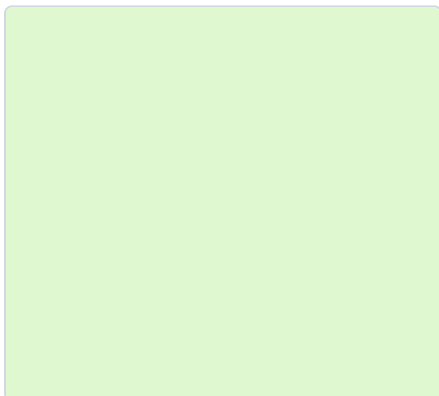
```
// ---- #include <paletteLutins> : les nuances des lutins ----  
  
#include <paletteLutins>    // paletteLutins() : les nuances d'une palette des lutins  
#include <sprite>  
#include <ALPHABET>
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

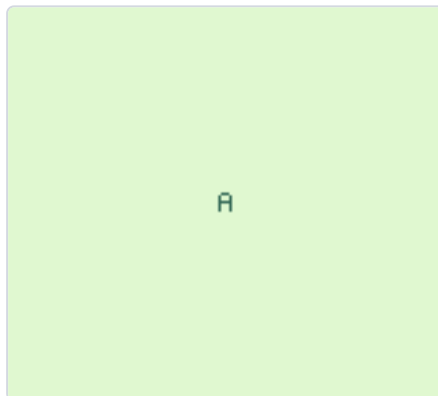
```
int main() {  
    paletteLutins(0, 0, 1, 2, 2);    // palette 0 : la teinte 3 prend la nuance 2, plus claire  
    sprite(0, 76, 68, ALPHABET[0]);  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un A gris foncé plutôt que noir, au milieu de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le registre de la palette 0 des lutins vaut 0, 1, 2, 2 (164)

Ensuite : **99. Les lutins : trois nuances et la transparence** — chapitre 15 : Les quatre nuances : la Game Boy d'origine. · gameboy3 — écrire une cartouche Game Boy en C++